

Outdoor VLA Robot Project

4 인 역할분담 및 실행계획 요약본

LiDAR/Depth-free Outdoor Semantic Navigation
센서: RGB Camera + IMU + GPS/GNSS / 메인: CARLA + ROS2

최종 방향	큰 실외 맵에서 보행자 30~50 명이 움직이고, 로봇이 RGB/IMU/GNSS 기반으로 위치와 주변 위험도를 추정해 VLA 정책에 따라 이동한다.
처음 MVP	CARLA + ROS2, 보행자 10 명, actor 기반 risk map, mock VLA policy, custom holonomic controller 부터 연결한다.
나중 확장	YOLO 검출, 실제 VLA API, 보행자 30~50 명, Unity 발표 시각화, MuJoCo 제어 검증을 단계적으로 추가한다.

주의: 이 문서는 팀원이 빠르게 읽고 바로 작업을 나누기 위한 실행 요약본이다. 자세한 이론 설명보다 역할, 툴, 산출물, 순서를 우선한다.

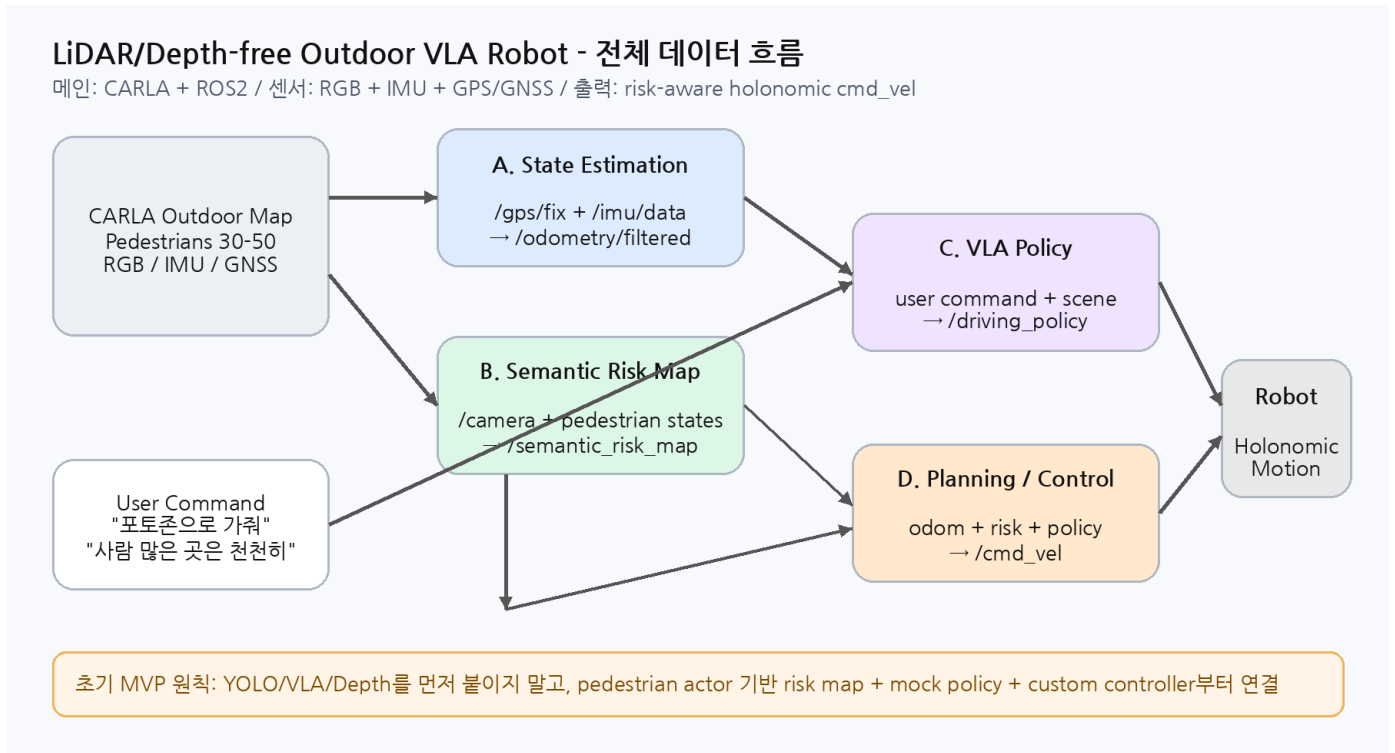
1. 전체 툴 스택 - 무엇을 쓸지

구분	선택	이유	담당
메인 시뮬레이터	CARLA	큰 실외 맵, 보행자 actor, RGB/IMU/GNSS 구성에 가장 적합	공통
ROS 연동	carla_ros_bridge	CARLA 센서/actor 정보를 ROS2 topic 으로 연결	A/B/D
미들웨어	ROS2 Humble	기존 개발 환경과 호환, 토픽 기반 역할 분리 용이	공통
상태추정	robot_localization + navsat_transform_node	GPS/GNSS + IMU 를 odom/world frame 으로 변환	A
비전	YOLO11 또는 YOLOv8 + OpenCV	RGB 에서 사람/장애물 검출. 초기에는 actor GT 로 대체	B
Risk Map	Python ROS2 node + NumPy + OccupancyGrid	사람 위치/밀도/위험구역을 cost 로 변환	B
VLA/정책	LLM/VLM API + JSON Schema	자연어 명령을 planner 가 읽을 수 있는 policy 로 변환	C
주행/제어	Custom Holonomic Controller, 이후 Nav2 참고	초기 통합이 빠르고 홀로노믹 vx/vy/wz 반영 쉬움	D
시각화	RViz2 + Foxglove	경로, risk map, 토픽 상태를 팀원이 같이 확인	공통
보조 시각화	Unity Robotics Hub	발표용 공원/캐릭터/NPC 연출 후보. 필수 아님	선택
보조 제어	MuJoCo	홀로노믹 바퀴 제어식 검증용. 메인 아님	D 선택

핵심 선택 기준

- 큰 실외 맵과 많은 보행자가 우선이면 CARLA 를 메인으로 둔다.
- Unity 는 공원/캐릭터 비주얼에는 좋지만 GPS/IMU/ROS2 로봇 기능을 직접 구현해야 하므로 보조 후보로 둔다.
- MuJoCo 는 큰 실외 맵/군중용이 아니라 홀로노믹 제어 검증용으로만 쓴다.
- 처음에는 YOLO/VLA/Depth Anything 없이 actor 기반 risk map + mock policy 로 시스템 뼈대를 먼저 붙인다.

2. 전체 구조 - 각 파트가 어떻게 연결되는지



전체 데이터 흐름은 CARLA 센서/보행자 → A/B/C/D ROS2 노드 → /cmd_vel → 로봇 이동 순서다. 각 담당자는 자기 출력 토픽만 맞추면 다른 파트와 병렬 개발할 수 있다.

입력	중간 처리	출력	사용자
/gps/fix, /imu/data	A: 상태추정	/odometry/filtered, /robot_global_pose	D, C
/camera/image_raw, /pedestrian_states	B: 위험도 생성	/semantic_risk_map, /risk_markers	C, D
/user_command, scene summary	C: 정책 생성	/driving_policy, /behavior_mode	D
/odometry/filtered, /semantic_risk_map, /driving_policy	D: 계획/제어	/planned_path, /cmd_vel	CARLA Robot

3. 4인 역할 카드 - 각자 무엇을 해야 하는지

A. Outdoor State Estimation

목표: GPS/GNSS + IMU 기반 로봇 위치와 자세 추정

입력: /gps/fix, /imu/data

출력: /odometry/filtered, /robot_global_pose, /gps_path

툴: robot_localization, navsat_transform_node, tf2_ros, RViz2

첫 완료 기준: CARLA 에서 GNSS/IMU 토픽을 받고 /odometry/filtered 를 안정적으로 발행한다.

B. Semantic Vision / Risk Map

목표: RGB/보행자 actor 기반 사람, 장애물, 위험구역 인식 및 risk map 생성

입력: /camera/image_raw, /pedestrian_states, /robot_global_pose

출력: /semantic_risk_map, /risk_markers, /semantic_objects_3d

툴: YOLO, OpenCV, Python ROS2, NumPy, OccupancyGrid

첫 완료 기준: 처음에는 actor 위치 기반 risk map 을 만들고, 이후 YOLO 를 붙인다.

C. VLA / Policy Reasoning

목표: 자연어 명령을 주행 정책 JSON 으로 변환

입력: /user_command, /robot_global_pose, /semantic_risk_map_summary

출력: /driving_policy, /target_request, /behavior_mode

툴: LLM/VLM API, JSON Schema, rclpy

첫 완료 기준: 처음에는 rule-based mock policy 로 시작하고, 마지막 에 실제 VLA API 로 교체한다.

D. Holonomic Planning / Control

목표: 전역 위치 + 위험도 + 정책을 반영해 실제 이동 제어

입력: /odometry/filtered, /semantic_risk_map, /driving_policy

출력: /planned_path, /cmd_vel, /robot_state

툴: Custom controller, Nav2 참고, MuJoCo 선택

첫 완료 기준: 목표점 이동, 감속/정지, 위험구역 회피, 홀로노믹 vx/vy/wz 제어를 구현한다.

4. 공통으로 해야 할 일

공통 작업	내용	완료 기준
GitHub/브랜치	공통 repo 생성, 패키지별 폴더 분리, 각자 branch 사용	main 에 기본 README 와 패키지 구조 존재
토픽 이름 고정	/gps/fix, /imu/data, /odometry/filtered, /semantic_risk_map, /driving_policy, /cmd_vel 고정	rqt_graph 에서 토픽 이름 불일치 없음
메시지 포맷 합의	policy JSON 필드, pedestrian state, risk map 규칙 정의	샘플 메시지 1 개씩 문서화
성능 제한	camera 10-15 FPS, YOLO 5 FPS, risk map 10 Hz, VLA 이벤트 기반	팀원 PC 에서 지연 없이 실행
시각화	RViz2/Foxglove layout 공유	risk map, path, robot pose, markers 표시
기록	rosviz2 는 필요한 토픽만 저장	재현 가능한 시연 bag 1 개 확보

처음 1 주차 목표

담당	이번 주에 끝낼 것	확인 명령/화면
A	CARLA GNSS/IMU 토픽 수신, /odometry/filtered 발행	ros2 topic echo /gps/fix, RViz2 pose
B	보행자 10 명 spawn, /pedestrian_states 발행, actor 기반 risk marker 표시	RViz2 MarkerArray
C	명령 5 개를 고정 policy JSON 으로 변환, /driving_policy 발행	ros2 topic echo /driving_policy
D	목표점 3 개 정의, /cmd_vel 이동, risk 높으면 감속/정지	로봇 이동 영상, /cmd_vel

5. 단계별 로드맵 - 무엇부터 할지

단계	목표	주요 작업	완료 기준	담당
Phase 1	CARLA + ROS2 연결	CARLA 실행, carla_ros_bridge 연결, RGB/IMU/GNSS 토픽 확인	/camera/image_raw, /imu/data, /gps/fix 확인	A/D
Phase 2	보행자 생성	walker 10명 spawn, 랜덤 이동, /pedestrian_states 발행	보행자 이동 + 위치 토픽 확인	B
Phase 3	상태추정	robot_localization, navsat_transform_node 설정	/odometry/filtered + /tf 정상	A
Phase 4	Risk Map 1 차	actor 위치 기반 local risk map 생성	사람 이동에 따라 /semantic_risk_map 변화	B
Phase 5	Mock VLA	rule-based policy JSON 생성	명령 입력 시 /driving_policy 변화	C
Phase 6	제어 1 차	목표점 이동, risk 기반 감속/정지	사람 주변 1.5m 정지, 3m 감속	D
Phase 7	RGB Vision	YOLO 기반 사람 검출 추가	bbox 표시, risk map 보정	B
Phase 8	VLA API	mock policy를 실제 API 기반으로 교체	자연어 명령이 policy로 변환	C
Phase 9	확장/발표	보행자 30~50명, Foxglove/RViz 화면 정리	최종 시나리오 영상 확보	공통

확장 순서

- 보행자는 10명 → 30명 → 50명 순서로 늘린다. 80명 이상은 도전 목표다.
- 처음부터 YOLO/VLA/Depth Anything을 붙이지 않는다. 통합 뼈대가 먼저다.
- 큰 맵 전체를 costmap으로 계산하지 않는다. 로봇 주변 20m x 20m local risk map만 계산한다.
- 발표 안정 버전은 보행자 30명 + actor 기반 risk map + mock/VLA policy + GPS 목표점 이동으로 잡는다.

6. 토픽 계약 - 서로 맞춰야 할 인터페이스

토픽	타입 후보	발행	구독	설명
/camera/image_raw	sensor_msgs/Image	CARLA	B	RGB 카메라 입력
/gps/fix	sensor_msgs/NavSatFix	CARLA	A	GPS/GNSS 전역 위치
/imu/data	sensor_msgs/Imu	CARLA	A	자세, 각속도, 가속도
/pedestrian_states	custom 또는 MarkerArray	B 또는 bridge	B/C/D	보행자 위치, 속도, ID
/odometry/filtered	nav_msgs/Odometry	A	C/D	필터링된 로봇 pose
/robot_global_pose	geometry_msgs/ PoseStamped	A	C/D	전역 pose 요약
/semantic_risk_map	nav_msgs/OccupancyGrid	B	C/D	사람/위험구역 기반 local cost
/risk_markers	visualization_msgs/ MarkerArray	B	공통	RViz/Foxglove 시각화
/driving_policy	std_msgs/String 또는 custom	C	D	VLA/정책 JSON
/target_request	custom 또는 PoseStamped	C	D	목표 구역/목표점 요청
/planned_path	nav_msgs/Path	D	공통	계획 경로
/cmd_vel	geometry_msgs/Twist	D	CARLA robot	홀로노믹 이동 명령

초기 policy JSON 필드

필드	예시	의미
action	go_to_zone / stop / follow_person	행동 모드
target_zone	photo_spot_A	목표 구역
max_speed	0.35	최대 속도
safety_margin	1.5	사람과 유지할 최소 거리
risk_weight	2.0	위험도 반영 가중치
avoid_zones	["parade_area"]	회피할 구역
behavior_mode	normal / caution / emergency_stop	전체 주행 모드

7. 체크리스트 - 각자 끝났다고 말할 기준

담당	1 차 완료 기준	2 차 완료 기준
A	/gps/fix, /imu/data, /odometry/filtered 정상 발행	/tf 안정, gps_path 표시, D 가 pose 사용 가능
B	보행자 10 명 actor 기반 /semantic_risk_map 생성	YOLO 검출 결과로 risk map 보정, 보행자 30 명 이상 처리
C	명령 5 개가 고정 policy JSON 으로 변환	VLA API 연결, scene summary 기반 policy 생성
D	목표점 이동 + risk 기반 감속/정지	보행자 30~50 명에서 안정 이동, path/risk/policy 모두 반영
공통	CARLA + ROS2 bringup 한 번에 실행	최종 영상, rosbag, 발표 화면 정리

하지 말아야 할 것

하지 말 것	이유	대신 할 것
처음부터 Isaac Sim 풀환경	무겁고 세팅 리스크 큼	CARLA 로 MVP 먼저 구성
처음부터 YOLO + VLA + Depth 동시 실행	디버깅 불가능, FPS 저하	actor risk map + mock policy 먼저
큰 맵 전체를 costmap 으로 계산	연산량 과다	로봇 주변 local risk map 만 계산
VLA 가 직접 /cmd_vel 생성	안전성/재현성 낮음	VLA 는 policy 만 생성, D 가 제어
보행자 100 명부터 시작	성능 문제 원인 추적 어려움	10 명 → 30 명 → 50 명 순서
토픽 이름을 각자 다르게 사용	통합 지연	토픽 계약표 기준으로 고정

최종 시연 시나리오

순서	장면	보여줄 기능
1	큰 실외 맵에서 보행자들이 이동	CARLA crowd scene
2	사용자: "포토존 A 로 가줘"	C: VLA/policy 생성
3	로봇이 GPS 목표점 방향으로 이동	A+D: pose + control
4	사람이 가까워짐	B+D: risk map 기반 감속/정지
5	사용자: "사람 많은 곳은 천천히 지나가"	C+D: max_speed/risk_weight 반영
6	포토존 근처 정지	최종 목표 도달